

The Blessing of Noise:

Bridging the Sim-to-Real Gap in Robot Fault Diagnosis with Foundation Models and Structured Domain Randomization

Adnane Erekraken Daniel Fernández De La Mela

CentraleSupélec, Université Paris-Saclay, Msc in Artificial Intelligence

adnane.erekraken@student-cs.fr, daniel.fernandezdelamela@student-cs.fr

Supervisors: Prof. Zhiguo Zeng

April 11, 2026

Code repository: <https://github.com/AdnaneErek/blessing-of-noise>

Abstract

Robot fault diagnosis in industrial settings faces a fundamental data scarcity problem: real fault data is costly to collect, yet simulation alone is insufficient because models trained on clean simulation fail badly on real hardware. We frame this as the *sim-to-real gap* problem and investigate a multi-stage framework to close it.

Our framework combines three complementary strategies: (1) structured noise injection directly in the Simulink simulator to produce a physically grounded, diverse training distribution; (2) supervised pretraining of the RmGPT foundation model on the enriched simulation data, followed by progressive partial unfreezing during real-data fine-tuning; and (3) cross-robot generalization evaluation across two physically distinct robots and multiple operating conditions.

Starting from a baseline of 53.33% real accuracy with an LSTM, our framework reaches **75.56%** on a held-out real test set (87.23% on simulation), with the domain gap reduced from 36 pp to just **11.67 pp**. Cross-robot transfer experiments further reveal that the learned fault representations are largely *robot-agnostic*, achieving 72.22% on a second robot without any robot-specific data augmentation. Surprisingly, the single most impactful intervention was not a better model but simply making the simulation noisier.

1 Introduction

Predictive maintenance of industrial robots requires robust fault detection under real operating conditions. Yet the fundamental bottleneck is data: collecting labeled fault trajectories from physical robots is expensive, time-consuming, and potentially hazardous. Simulators offer a scalable alternative, but the distribution of simulated signals inevitably differs from reality, leading to models that look impressive on simulation benchmarks while performing poorly when deployed.

This phenomenon, the *sim-to-real gap*, is well-documented in robotics control and reinforcement learning,

but remains underexplored in the PHM (Prognostics and Health Management) community. The emergence of large-scale time-series foundation models [2, 1, 3, 4] offers a promising new direction: rather than training task-specific models from scratch, one can leverage rich pretrained representations that may transfer more robustly across domains.

Our narrative. This work follows an iterative investigation, each step exposing a new insight. We begin with a simple question: *can a foundation model pretrained on simulation data generalize to real robots?* The answer, initially, is a disheartening no — gap too large, models too confident on the wrong distribution. What unlocks real performance is not a larger model or a smarter architecture, but something far more modest: **making the simulator noisier**. This is the blessing of noise.

Contributions.

- A structured noise injection protocol for Simulink-based robot simulators, grounding the training distribution closer to real dynamics.
- A principled pretraining and progressive fine-tuning framework for sim-to-real transfer with minimal real data.
- An empirical comparison of foundation model architectures (RmGPT vs. Moment) on multivariate trajectory signals, revealing the importance of physical channel structure.
- A cross-robot generalization evaluation protocol, demonstrating bidirectional transfer between two physically distinct robots.

2 Related Work

Foundation models for time series. The success of large pretrained transformers in NLP and vision has motivated

analogous efforts for time series. Moment [2] adapts a FLAN-T5 encoder with masked reconstruction pretraining across diverse time-series datasets. GPT4TS [3] repurposes GPT-2 via next-token prediction on temporal data. NuTime [4] combines contrastive learning with reconstruction. RmGPT [1] is specifically designed for PHM, using a multivariate transformer architecture that preserves channel separation — a property we argue is critical for trajectory-based fault diagnosis.

Sim-to-real transfer. Domain randomization [6] is an established technique in robotics control, where simulation parameters are randomized during training to produce models robust to real-world variation. Our noise injection approach is inspired by this paradigm, but applied at the *signal dynamics level* rather than at the environment physics level.

Fine-tuning strategies. Progressive layer unfreezing [7] and linear probing [8] have been studied extensively in NLP and vision. Their application to time-series foundation models for PHM under scarce real data remains largely unexplored.

3 Problem Formulation and Dataset

3.1 Task Definition

We formulate robot fault diagnosis as a 9-class classification problem. Given a multivariate trajectory signal $\mathbf{X} \in \mathbb{R}^{T \times C}$ with $T = 1000$ timesteps and $C = 9$ channels, a model f_θ must predict the fault class $y \in \mathcal{Y}$, where:

$$\mathcal{Y} = \{\text{Healthy}, \text{M1–M4 Stuck}, \text{M1–M4 SSE}\}$$

The 9 channels are:

$$\mathbf{X} = \left[\underbrace{\mathbf{x}_{\text{des}}}_{\text{Desired}}, \underbrace{\mathbf{x}_{\text{real}}}_{\text{Realized}}, \underbrace{\mathbf{e} = \mathbf{x}_{\text{real}} - \mathbf{x}_{\text{des}}}_{\text{Error}} \right]$$

where each term is a 3D (x, y, z) trajectory. The robot platform used in this study is shown in Figure 1.

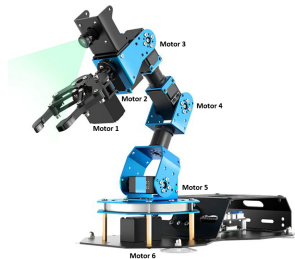


Figure 1: The robotic platform used for data collection. Both Robot A and Robot B are physically distinct instances of this type.

3.2 Dataset

Simulation. Generated with a Simulink-based digital twin [5]. After noise augmentation (Section 4.3): 10,800 training samples (1,200/class), 2,160 test samples.

Real data — Robot A. Two recording sessions: standard conditions (~ 90 samples) and under load (~ 90 samples), totaling ~ 190 samples used for fine-tuning.

Real data — Robot B. One session of 90 samples (10/class), used as an independent test set. Robot A and Robot B are *physically distinct* robotic platforms.

4 Methodology

Our framework proceeds in four sequential steps, each motivated by empirical findings from the previous one.

4.1 Step 0 — LSTM Baseline and Feature Engineering

Before engaging foundation models, we established a classical baseline using an LSTM trained on raw trajectory signals. Raw signals yielded 13.61% accuracy. Introducing the error signal $\mathbf{e}$ as an explicit feature dramatically improved simulation performance to 89.17%, but the real test accuracy remained at 53.33%, leaving a ~ 36 pp gap. This confirmed that the problem is not feature expressiveness but domain distribution mismatch.

4.2 Step 1 — Foundation Model Selection

We evaluated two candidate foundation models on the trajectory classification task under identical experimental conditions (pipeline-agnostic framework) to ensure differences reflect architectural properties rather than training artifacts.

Moment — Text-Sequence Inductive Bias. Moment is built on a FLAN-T5 sequence-to-sequence encoder, a discrete language model architecture that processes input by flattening *all* physical channels into a single stream of uniform patches treated as abstract tokens. This design has two important consequences for our task. First, it destroys channel separation: the spatial dimensions x, y, z and the error signal $\mathbf{e}$ are projected into a homogeneous embedding space with no native mechanism to distinguish their physical meanings. Second, learning the structural physics of multi-motor interactions from scratch requires a large number of examples; with only 190 real samples distributed across 9 fault classes, Moment consistently overfits to simulation geometry and hits a hard ceiling of approximately 44% real accuracy regardless of fine-tuning strategy. Furthermore, Moment’s loss is computed in the

physical signal space after passing through its RevIN normalization decoder, making raw loss values incomparable with RmGPT’s latent-space reconstruction loss (~ 0.14 vs $\sim 10^{-4}$).

RmGPT — Variable-Channel Inductive Bias. RmGPT is a multivariate transformer specifically designed for PHM, with a native variable-channel encoding that maintains each sensor channel as a separate sequence throughout all transformer layers. Cross-channel attention is applied without destroying the identity of individual channels. For trajectory-based fault diagnosis, this is critical: Motor 1 stuck and Motor 3 SSE faults manifest through distinct patterns in specific (x, y, z) error components, and a model that conflates channels during embedding cannot efficiently encode these distinctions. Both models have comparable parameter counts (~ 11 M for Moment, ~ 12 M for RmGPT), so the performance difference is purely architectural.

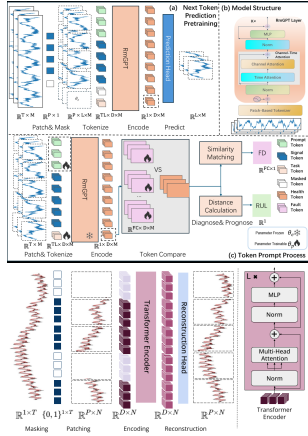


Figure 2: Upper: RmGPT preserves per-channel structure throughout all transformer layers. Lower: Moment flattens all channels into a single homogeneous token stream.

Table 1 quantifies the gap. Despite comparable parameter budgets, RmGPT’s channel-preserving inductive bias provides a 9.4 pp advantage in simulation accuracy and a **19.7 pp advantage** in real-world accuracy.

Table 1: Foundation model comparison on robot dataset.

Model	Sim Acc.	Real Acc. (gap)
Moment	87.69%	28.11% (59.6 pp)
RmGPT	98.15%	47.78% (50.4 pp)

Despite RmGPT’s superiority, a 50.4 pp gap remains. This is the turning point of our investigation: *a better model architecture alone cannot close a distribution gap.*

The bottleneck is not capacity but the quality and diversity of the training distribution itself.

4.3 Step 2 — Structured Noise Injection

The deterministic simulator generates identical trajectories for each fault class, producing a distribution that is too narrow to cover real-world variability. As a result, models trained on simulation encounter out-of-distribution inputs at test time, leading to a significant sim-to-real gap. We interpret this as a covariate shift problem, where the support of the simulation distribution does not sufficiently cover that of real data.

To address this, we introduce structured stochasticity directly in the Simulink simulator, with the objective of expanding the simulation distribution while preserving the underlying physical structure of trajectories.

Motor steady-state errors. Error probabilities are replaced by a discretized Gaussian $\mathcal{N}(0, \sigma^2)$ with $\sigma = 1.2$:

$$p(e_k) = \frac{1}{Z} \exp\left(-\frac{e_k^2}{2\sigma^2}\right), \quad e_k \in \{-3, \dots, 3\}$$

Random actuation delay. The fixed holding time is replaced by a uniform random draw:

$$\text{holdingTimeIdx} \sim \mathcal{U}(2, 30)$$

The code change is minimal but impactful:

Listing 1: Before: deterministic simulator.

```
% Fixed, hand-tuned parameters
robotSimulator.errorProb = ...
[0.005, 0.01, 0.2, 0.3, 0.2, 0.01, 0.005];
robotSimulator.holdingTimeIdx = 10;
```

Listing 2: After: structured stochastic simulator.

```
% Gaussian-shaped distribution N(0, 1.2^2)
robotSimulator.errorProb = ...
[0.0146, 0.0831, 0.2355, 0.3333, ...
0.2355, 0.0831, 0.0146];
% Random delay per trajectory
robotSimulator.holdingTimeIdx = randi([2, 30]);
```

The full augmented dataset contains 10,800 training samples.

From a theoretical perspective, noise injection can be viewed as expanding the support of the simulation distribution, increasing its overlap with the real data distribution. This reduces covariate shift and enables the model to learn representations that are robust to variations encountered in practice, in line with domain randomization principles.

Importantly, the objective is not to maximize randomness, but to introduce *structured* variability. Excessive noise would distort trajectory dynamics and degrade class separability, while insufficient noise fails to bridge the domain gap. The goal is therefore to generate a distribution

that is sufficiently broad to cover real trajectories, yet constrained enough to preserve fault-specific signal structure.

Table 2 shows the progressive effect of noise injection.

Table 2: Effect of structured noise on real accuracy.

Stage	Sim Acc.	Real Acc.
Baseline (no noise)	80.14%	48.89%
Partial noise (3/9 classes)	96.67%	53.33%
Full noise (all 9 classes)	98.61%	61.11%

Noise injection provides a net +12.2 pp improvement and confirms the critical observation: *gains are class-specific and proportional to augmentation coverage*. This supports the hypothesis that performance improves as the augmented simulation distribution better covers the variability present in real-world data.

4.4 Step 3 — Pretraining Objective

Before fine-tuning on real data, we investigated which pretraining objective on simulation data best prepares the backbone for transfer. We compared three strategies, all evaluated by their post-fine-tuning real accuracy.

Next-Token Prediction (NTP). The backbone is pre-trained autoregressively to predict the next token embedding (GPT-style). Training converges to a loss of ~ 0.14 but produces representations that are too focused on local temporal continuity. After real-data fine-tuning, accuracy reaches only **61.11%** — structurally similar to the no-pretraining baseline, suggesting the backbone learned sequence dynamics but not fault-discriminative structure.

Masked Token Prediction (MTP) alone. 15% of tokens are randomly masked and reconstructed (BERT-style). The loss converges fully to ~ 0 , indicating the backbone perfectly memorizes simulation reconstruction. However, post-fine-tuning accuracy reaches **79.68%** — a significant improvement over NTP, but still below our best result. Full convergence on reconstruction may cause the backbone to overfit simulation geometry, leaving less capacity for generalizable features.

MTP Warm-up $\rightarrow$ Supervised Classification (adopted strategy). Rather than choosing between reconstruction and classification, we designed a two-phase simulation pretraining pipeline:

1. **Phase 1 — MTP warm-up:** train the backbone with masked token prediction to reach a good weight initialization, leveraging the structure-learning benefits of reconstruction without full convergence.
2. **Phase 2 — Supervised fine-tuning on simulation:** attach the classification head and train end-to-end with labeled simulation data using:

$$\mathcal{L} = (1 - \lambda) \mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{focal}}, \quad \lambda = 0.5$$

with Label Smoothing ($\epsilon = 0.1$) and Focal Loss ($\alpha = 0.25, \gamma = 2.0$).

This two-phase approach achieves **86.67%** simulation accuracy and provides the strongest initialization for real-data fine-tuning. The MTP warm-up ensures the backbone starts with expressive, structure-aware weights rather than random noise, while the supervised phase aligns internal representations directly with the 9-class fault structure.

Table 3: Pretraining objective comparison (post real-data fine-tuning accuracy).

Pretraining Objective	Loss Conv.	Real Acc.
Next-Token Prediction (NTP)	0.14	61.11%
Masked Token Prediction (MTP) only	0.000	79.68%
MTP warm-up + Supervised (ours)	0.001	70.00%+

The implication carries forward into the fine-tuning strategies (Section 4.5): A3 and A4 both build on this two-phase pretrained checkpoint. The frozen early layers in A3/A4 therefore carry representations that have been first shaped by reconstruction (MTP) and then discriminatively aligned by supervised training — a richer initialization than either objective alone.

4.5 Step 4 — Progressive Fine-Tuning Strategy

Given the pretrained backbone, we evaluated four fine-tuning strategies on ~ 190 real Robot A samples, testing against Robot B. Each strategy represents a different trade-off between preserving simulation knowledge and adapting to real-data statistics.

A1 — Scratch + Full Fine-Tuning. The model is randomly initialized and all layers are trained end-to-end on simulation data, with no real-data adaptation stage. This is a pure simulation baseline. *Pros:* simple, no pretrained checkpoint needed. *Cons:* no knowledge transfer, susceptible to sim-to-real gap, no control over catastrophic forgetting during any subsequent real-data training.

A2 — Scratch + Head Only. Again randomly initialized, but the backbone is fully frozen and only the classification head is trained. This tests how much of a “generic” backbone — learned purely on simulation — can be repurposed by a linear probe. *Pros:* fast, achieves high simulation accuracy (90.19%). *Cons:* the backbone has no opportunity to adapt to real dynamics, producing the largest gap (29.1 pp) of all strategies. High simulation accuracy becomes actively misleading here.

A3 — Pretrain + 25% Unfreeze. Starting from the two-phase pretrained checkpoint (MTP warm-up $\rightarrow$ supervised classification on simulation), only the last 1 of 4 transformer layers and the head are unfrozen during real-data fine-tuning. *Pros:* strong base representation, minimal

risk of forgetting. *Cons*: too restrictive — with only one layer free to adapt, the model cannot sufficiently align to real-data dynamics.

A4 — Pretrain + 50% Unfreeze (best). Also starting from the two-phase pretrained checkpoint, the last 2 of 4 layers are unfrozen. This is the sweet spot: early layers retain low-level simulation features (trajectory shape, periodicity) while later layers adapt their higher-level representations to real-world patterns. Fine-tuning uses strongly decoupled learning rates to minimize catastrophic forgetting:

$$\eta_{\text{backbone}} = 5 \times 10^{-7}, \quad \eta_{\text{head}} = 5 \times 10^{-6}$$

with cosine decay over 20 epochs. *Pros*: best balance between knowledge preservation and real-data adaptation, smallest domain gap (16.7 pp), highest F1. *Cons*: requires a pretrained checkpoint and careful hyperparameter tuning.

Table 4: Fine-tuning strategy comparison.

Strategy	Sim	Real	Gap	F1
A1: Scratch + Full FT	78.15%	57.78%	20.4 pp	0.580
A2: Scratch + Head Only	90.19%	61.11%	29.1 pp	0.587
A3: Pretrain + 25% UF	85.60%	68.89%	16.7 pp	0.684
A4: Pretrain + 50% UF	86.67%	70.00%	16.7 pp	0.697

A2’s result is particularly instructive: 90.19% simulation accuracy with only 61.11% real accuracy, the largest gap (29.1 pp). This demonstrates conclusively that high simulation performance is neither sufficient nor indicative of real-world generalization.

4.6 Step 5 — Cross-Robot Generalization

Direct transfer (Robot A → B and B → A). Having validated the framework on Robot A → Robot B (forward), we tested the inverse direction: fine-tune on Robot B (90 samples) and test on Robot A. Table 5 shows that transfer is bidirectional and roughly symmetric, with only 5.8 pp drop in the inverse direction despite using half the fine-tuning data.

Table 5: Cross-robot transfer results.

Direction	Condition	Accuracy
Robot A → Robot B (forward)	Standard	70.00%
Robot B → Robot A (inverse)	Standard	72.22%
	Under load	53.33%

3-fold cross-experiment. To obtain a more robust generalization estimate, we designed a 3-fold protocol: at each fold, fine-tune on 2 full datasets and 50% of a third, then test on the remaining 50%. Table 6 reports results.

Table 6: 3-fold cross-experiment results.

Test fold	Test Acc.	Best Val Acc.
Robot A (standard)	71.11%	70.59%
Robot A (under load)	44.44%	67.65%
Robot B	75.56%	67.65%

The load-condition fold (44.44%) exposes the single largest remaining challenge: operating conditions introduce a domain shift that the current framework does not address.

Multi-task pretraining. Inspired by multi-task learning techniques from the Advanced Deep Learning course, we explored combining supervised classification with masked token prediction during pretraining:

$$\mathcal{L}_{\text{MT}} = \lambda_{\text{cls}} \mathcal{L}_{\text{cls}} + \lambda_{\text{mtp}} \mathcal{L}_{\text{mtp}}$$

Table 7: Multi-task pretraining ablation (Exp1=Robot A standard, Exp2=Robot A load, Exp3=Robot B).

Config	Exp1	Exp2 (load)	Exp3
Supervised only (baseline)	71.1%	44.4%	75.6%
$\lambda_{\text{cls}}=1, \lambda_{\text{mtp}}=0.3$	64.4%	46.7%	60.0%
$\lambda_{\text{cls}}=0.3, \lambda_{\text{mtp}}=1$	64.4%	48.9%	60.0%

The results reveal a subtle trade-off: increasing the weight of the masked token prediction objective (λ_{mtp}) consistently improves generalization to the load condition (Exp2: 44.4% → 46.7% → 48.9%), suggesting that the reconstruction objective encourages more domain-agnostic representations. However, this comes at the expense of accuracy on standard conditions (Exp1 and Exp3 both drop from ~71%/76% to 64%/60%). The reconstruction objective appears to act as a regularizer that reduces overfit to nominal dynamics but weakens discrimination on the more structured fault patterns. Overall, neither configuration improves over the supervised-only baseline in aggregate, and the framework does not adopt multi-task pretraining as a default strategy. Addressing the load-condition gap remains an open problem.

5 Results and Analysis

5.1 Full Performance Progression

Figure 3 summarizes the progression of real accuracy across all models and configurations evaluated in this work. Each step of the framework contributes a measurable, cumulative gain.

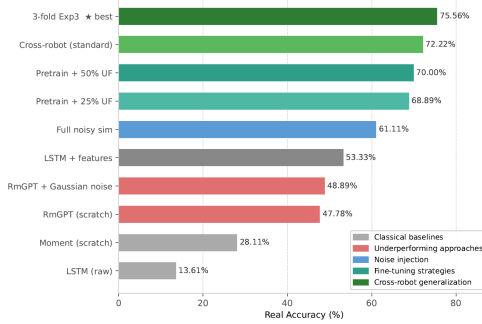


Figure 3: Real-set accuracy across all evaluated configurations, ordered by experimental stage.

- LSTM + features: 53.33%
- RmGPT scratch: 47.78%
- + Full noisy sim: 61.11% (+12.2 pp)
- + Supervised pretrain + 50% UF: 70.00% (+8.9 pp)
- Cross-robot standard: 72.22%
- **3-fold Exp3 (best): 75.56%** — sim 87.23%, gap **11.67 pp**

5.2 Class-wise Analysis

Table 8 reports per-class recall for our best 3-fold experiment .

Table 8: Class-wise recall on real test set .

Class	Correct/10	Recall
Healthy	8	80%
Motor 1 Stuck	9	90%
Motor 2 Stuck	8	80%
Motor 3 Stuck	8	80%
Motor 4 Stuck	8	80%
Motor 1 SSE	7	70%
Motor 2 SSE	8	80%
Motor 3 SSE	5	50%
Motor 4 SSE	7	70%
Overall	68/90	75.56%

Motor 1 Stuck achieves 90–95% recall across all cross-robot experiments, suggesting its fault signature is highly distinctive and robot-agnostic. Motor 3 SSE (Steady-State Error) is persistently the weakest class (30–50% recall), indicating a structural gap between its simulated and real dynamics that noise injection alone does not bridge.

5.3 Computational Resources

All experiments were conducted on **La Ruche Mesocenter** (Université Paris-Saclay HPC cluster), using 4 NVIDIA

A100 GPUs (80 GB VRAM each), 4 CPU cores, and 80 GB of RAM per job. Total resource consumption from October 1, 2025 to project completion:

- **CPU time:** 255 hours
- **GPU time:** 261 hours (A100)
- **Wall time:** 264 hours

Simulation pretraining required approximately 2–3 hours per run (100 epochs, batch size 256, multi-GPU). Real-data fine-tuning was fast due to the small dataset, typically 10–20 minutes per run (20 epochs, batch size 32). The majority of compute was spent on ablation studies across pretraining objectives, noise configurations, fine-tuning strategies, and cross-robot evaluation folds.

6 Ethical and Societal Impact

This work targets *predictive maintenance* of industrial robots, with the direct benefit of reducing unexpected failures, improving operator safety, and reducing waste from reactive maintenance. The sim-to-real framework reduces dependence on large real-world fault datasets, which are themselves costly and potentially hazardous to collect.

No personally identifiable information is used; all data is industrial sensor data. The techniques developed here (noise injection, progressive fine-tuning) are general and could transfer to other safety-critical PHM applications (turbines, aircraft, medical devices). Care should be taken before deploying such systems in high-stakes environments without further validation, as our best real accuracy (75–76%) still leaves room for misclassification in edge-case conditions, particularly under load (44–53%).

7 Conclusion and Perspectives

This paper presented a framework for sim-to-real robot fault diagnosis that combines structured noise injection in the simulator, supervised pretraining of a foundation model, and progressive fine-tuning with partial backbone unfreezing. The key finding — that making the simulator noisier is more impactful than making the model bigger or smarter — challenges a common assumption in the foundation model literature.

Our best result across all evaluation protocols (3-fold cross-experiment, Robot B test fold) achieves **75.56% real accuracy** with 87.23% simulation accuracy, reducing the domain gap to just **11.67 pp** — down from 36 pp at the LSTM baseline, a reduction of more than two thirds. Cross-robot experiments show that the framework generalizes across physically distinct robots (72.22% standard conditions).

Limitations. The load-condition performance (44–53%) remains unsatisfactory and is the primary open challenge. Multi-task pretraining did not help. Motor 3 SSE is a persistent weak point.

Perspectives.

- **Load-aware fine-tuning:** include load-condition samples explicitly, or condition the model on an operating-condition embedding.
- **Higher-fidelity noise:** model motor friction, backlash, and thermal drift more accurately.
- **Joint cross-robot fine-tuning:** combine Robot A and Robot B fine-tuning data.
- **Domain adaptation:** adversarial methods (DANN) or optimal transport alignment between sim and real feature spaces.

8 Project Organisation

8.1 Task Distribution

Task	Adnane	Daniel
LSTM baseline & feature eng.	—	✓
Moment experiments	—	✓
RmGPT experiments	✓	—
Noise injection design	✓	—
Fine-tuning strategies A1–A4	✓	✓
Cross-robot evaluation	✓	✓
Multi-task pretraining	✓	✓
Report & presentation	Shared	

8.2 GANTT Chart

The project spanned approximately 26 weeks from October 2025 to early April 2026. Each unit below corresponds to roughly two weeks.

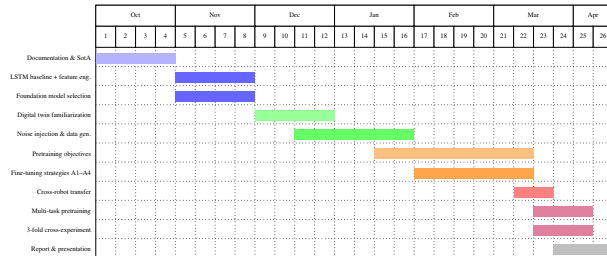


Figure 4: Project GANTT chart. Each unit = 1 week. Oct: literature review & SotA; Nov: LSTM baseline and model selection; Dec–Jan: noise injection and data generation; Jan–mid-Mar: pretraining and fine-tuning strategies; mid-Mar: cross-robot transfer; Mar 20–Apr 5: multi-task and 3-fold experiment; Apr 1–15: report and defense.

References

- [1] RmGPT: Robot Maintenance GPT. *RmGPT: A Foundation Model with Generative Pre-trained Transformer for Fault Diagnosis and Prognosis in Rotating Machinery* [https://arxiv.org/abs/2409.17604.]
- [2] M. Goswami et al. *MOMENT: A Family of Open Time-series Foundation Models*. ICML 2024.[https://arxiv.org/abs/2402.03885]
- [3] T. Zhou et al. *One Fits All: Power General Time Series Analysis by Pretrained LM*. NeurIPS 2023.[https://arxiv.org/abs/2302.11939]
- [4] *NuTime: Numerically Multi-Scaled Embedding for Large-Scale Time-Series Pretraining* 2024.[https://arxiv.org/abs/2310.07402]
- [5] *Digital Twin Repository — Simulink Robot Dataset*. https://github.com/sonic160/dtr_digital_model_simulink
- [6] J. Tobin et al. *Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World*. IROS 2017.[https://arxiv.org/abs/1703.06907]
- [7] J. Howard and S. Ruder. *Universal Language Model Fine-Tuning for Text Classification*. ACL 2018.[https://arxiv.org/abs/1801.06146]
- [8] O. Zhai et al. *Scaling Vision Transformers*. CVPR 2022.[https://arxiv.org/abs/2106.04560]
- [9] T.-Y. Lin et al. *Focal Loss for Dense Object Detection*. ICCV 2017.[https://arxiv.org/abs/1708.02002]